LAPORAN PRAKTIKUM TEKNOLOGI IOT 2

MINGGU-3

Oleh :

Nama : Gohan Kristian Pangaribuan

NPM : 25782040

Kelas : TRI (3B)

Dosen pengampu : Dr.Ir. Septafiansyah Dwi Putra, S.T.,M.T.

Agus ambarwari, S.Pd., M.Kom.

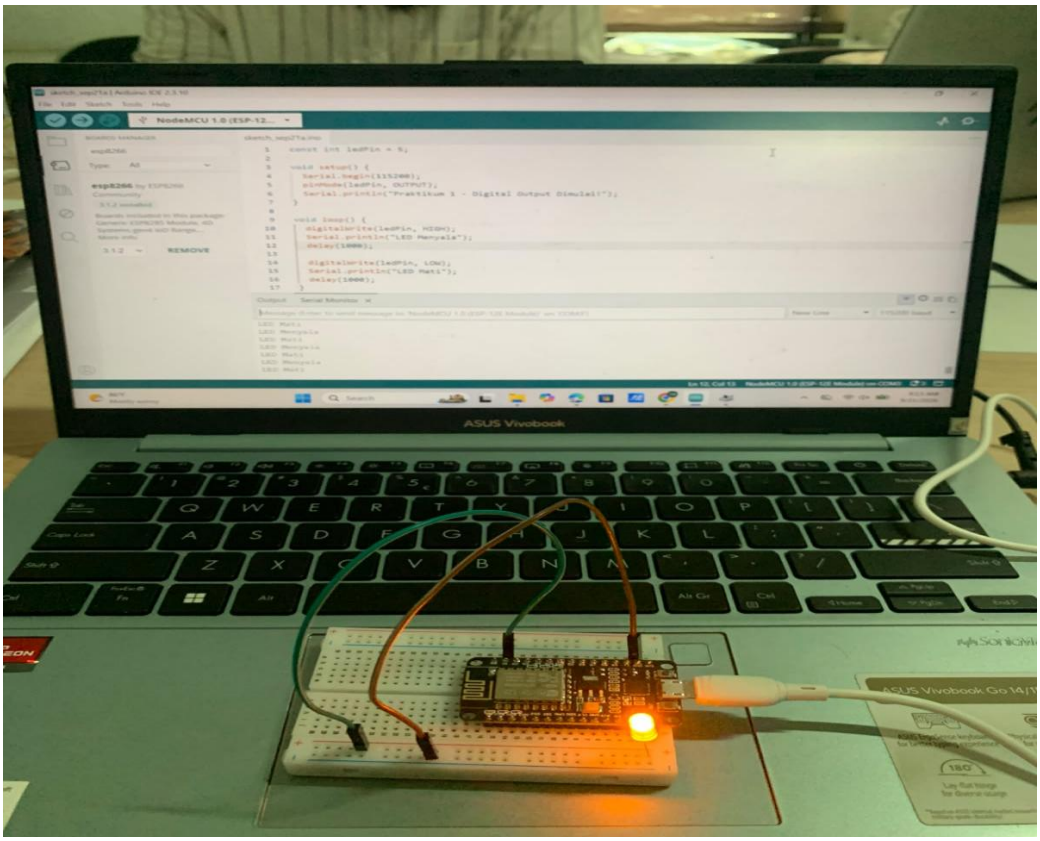
JURUSAN TEKNOLOGI INFORMASI

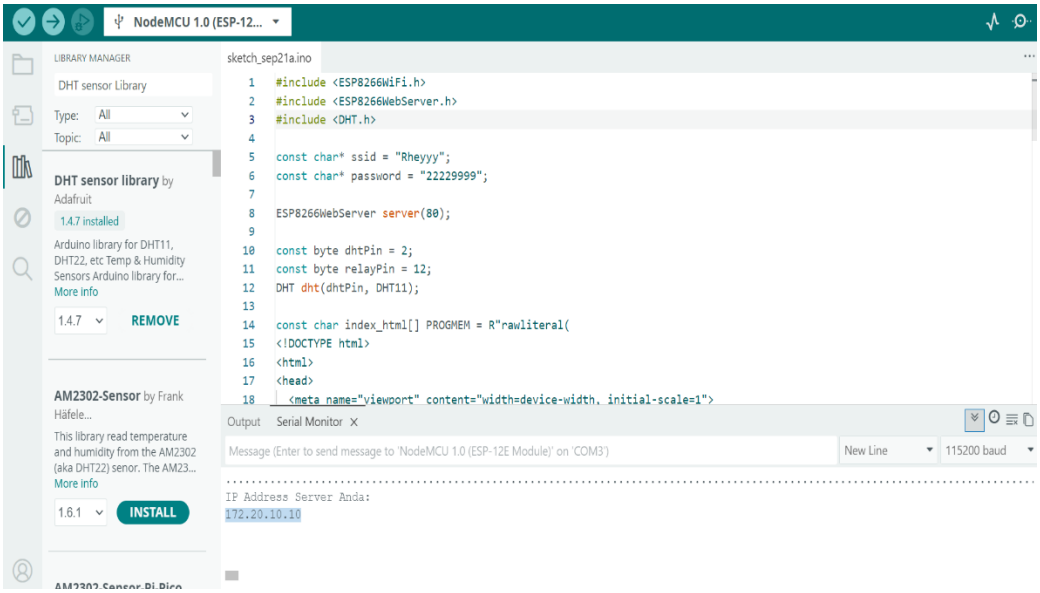
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

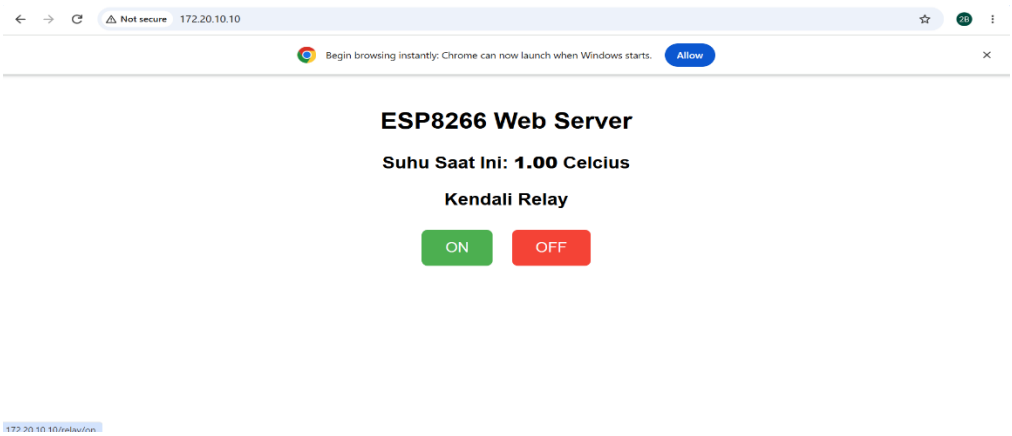
POLITEKNIK NEGERI LAMPUNG

2025/2026

3.5.1 Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)







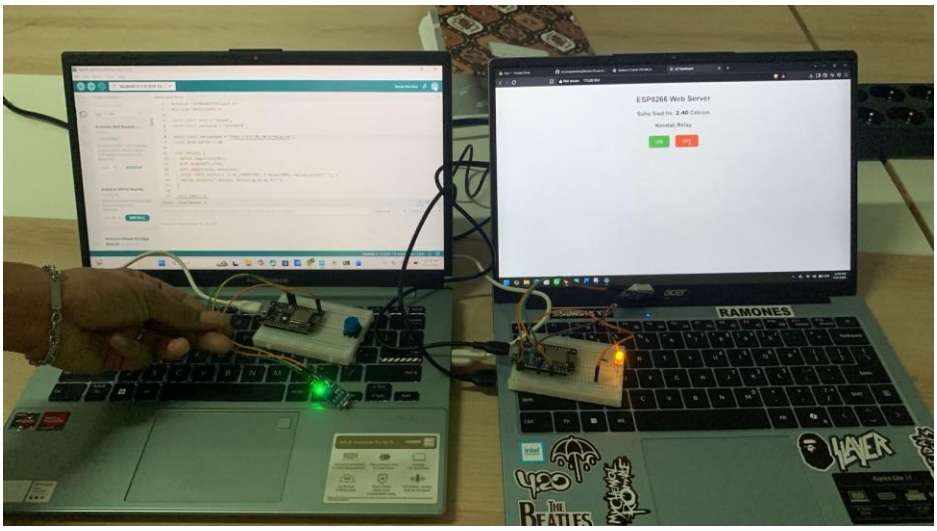
Langkah Kerja Rangkaian:

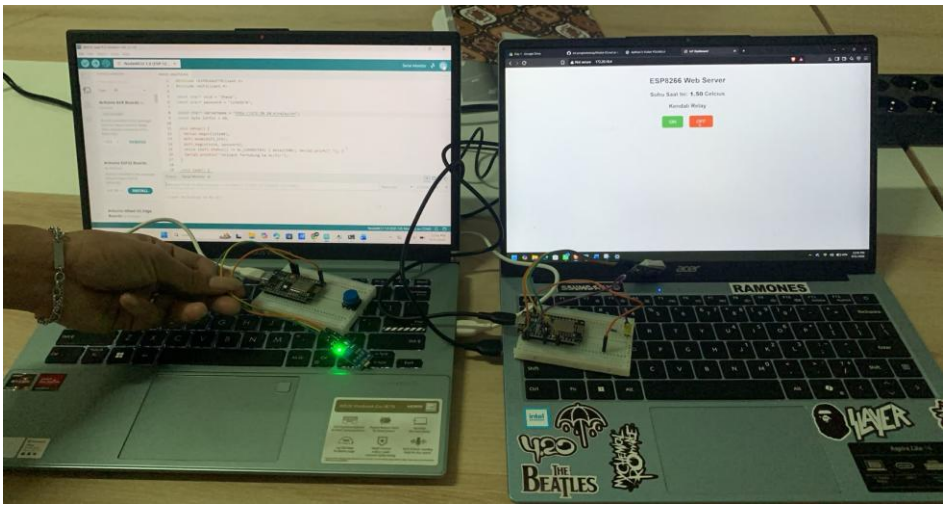
1. Hubungkan pin DATA / OUT / S DHT ke pin D4 (GPIO 2). (Pastikan VCC DHT ke 3V3, GND ke GND).
2. Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12). Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke pin GND NodeMCU. *(Catatan: LED ini dikendalikan oleh pin D6 / GPIO 12 yang pada kode program didefinisikan sebagai relayPin).*

Penjelasan Singkat Kode:

- ESP8266WebServer server(80);: Menginisialisasi *library Server* agar terus mendengarkan panggilan di jalur lalu lintas standar web (*Port 80*).
- html.replace("%TEMPERATURE%", String(t));: Fitur manipulasi teks (*String*). Perintah ini secara cerdas akan menyapu dokumen HTML, mencari teks *placeholder* %TEMPERATURE%, dan menggantinya dengan variabel angka suhu yang didapat dari sensor aktual sebelum dikirim ke layar pengunjung. Inilah esensi dari halaman *web* dinamis.
- server.on("/relay/on", handleRelayOn);: Fungsi *Routing*. Jika ada klien mengunjungi tautan URL berakhiran /relay/on, server akan menjalankan aksi mengubah *state* LED (menyalakan/mematikan output), lalu me-*redirect* layar pengunjung kembali ke halaman depan (Location: /) via kode 303.
- server.handleClient();: Berada di dalam loop() untuk terus-menerus mengecek apakah ada permintaan pengunjung yang datang setiap milidetiknya.

3.5.2 Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)





sketch_sep21a.ino

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>

const char* ssid = "dhewa";
const char* password = "12345678";

const char* serverName = "http://172.20.10.4/relay/on";
const byte ldrPin = A0;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nClient Terhubung ke Wi-Fi!");
}
```

Output Serial Monitor ×

Not connected. Select a board and a port to connect automatically.

New Line

Langkah Kerja Rangkaian:

1. Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin A0.

Penjelasan Singkat Kode:

- WiFiClient client; dan HTTPClient http;: Kelas pemanggil untuk memulai eksekusi permintaan *client* jarak jauh (mirip fungsi aplikasi *browser* di laptop).
- http.begin(client, serverName);: Menargetkan sasaran URL *server*.
- int httpResponseCode = http.GET();: Mengeksekusi ketukan pintu (*Request* HTTP GET). Jika Server merespons sukses, nilai variabel ini akan berisi angka standar jaringan seperti 200 (OK) atau 303 (Redirect). Jika URL rusak atau *Server offline*, variabel akan mereturn nilai negatif.
- delay(10000);: *Cooldown* panjang 10 detik ini ditambahkan agar *Client* tidak memborbardir (*flood/DDoS*) Server dengan permintaan on secara terus-menerus setiap detik ketika kondisinya sedang gelap

3.6 Latihan

Latihan 1: Pemahaman Jaringan

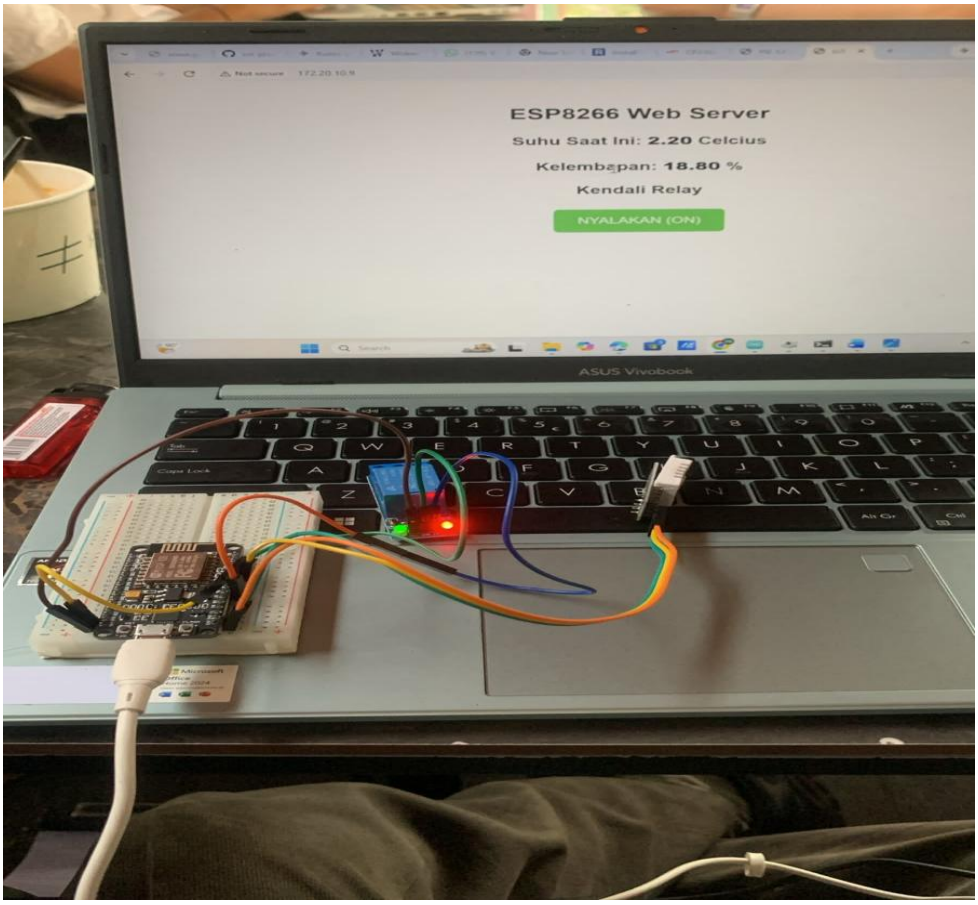
Pertanyaan: Mengapa kita menggunakan mode jaringan Station (STA) pada kedua praktikum, dan bukan mode Access Point (AP)?

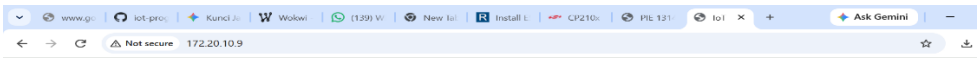
Jawaban:

- Terhubung ke Jaringan Bersama: Mode STA (*Station*) membuat NodeMCU bertindak sebagai *client* yang terhubung ke jaringan Wi-Fi yang sama (seperti *router* atau *hotspot* lokal). Hal ini memungkinkan mikrokontroler dan perangkat pengontrol (laptop/smartphone) berada dalam satu segmen jaringan lokal yang sama sehingga mudah saling berkomunikasi.
- Aksesibilitas dan Infrastruktur: Jika menggunakan mode AP (*Access Point*), NodeMCU akan memancarkan sinyal Wi-Fi sendiri. Akibatnya, perangkat yang terhubung ke NodeMCU akan kehilangan koneksi internet atau jaringan utama lokal, dan hanya bisa berkomunikasi secara terbatas dengan satu perangkat tersebut. Oleh karena itu, mode STA lebih ideal untuk sistem IoT berbasis infrastruktur jaringan yang ada.

Latihan 2: Modifikasi Antarmuka & Telemetri

Untuk menampilkan kelembapan dan membuat tombol *toggle* dinamis tunggal, berikut adalah modifikasi kode pada HTML dan fungsi `handleRoot()`:





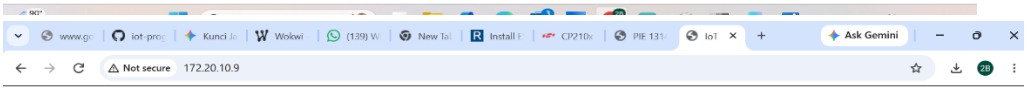
ESP8266 Web Server

Suhu Saat Ini: 1.00 Celcius

Kelembapan: 19.00 %

Kendali Relay

NYALAKAN (ON)



ESP8266 Web Server

Suhu Saat Ini: 1.00 Celcius

Kelembapan: 18.20 %

Kendali Relay

MATIKAN (OFF)

```
27  </head>
28  <body>
29    <h1>ESP8266 Web Server</h1>
30    <div class="sensor-box">
31      <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
32      <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
33    </div>
34    <h2>Kendali Relay</h2>
35    %RELAY_BUTTON%
```

```
40  void handleRoot() {
41    String html = index_html; // Salin kerangka HTML ke variabel dinamis
42    float t = dht.readTemperature();
43    float h = dht.readHumidity();
44
45    // Mengganti teks placeholder %TEMPERATURE% dengan suhu nyata
46    if (isnan(t)) {
47      html.replace("%TEMPERATURE%", "--");
48    } else {
49      html.replace("%TEMPERATURE%", String(t));
50    }
51
52    // Mengganti teks placeholder %HUMIDITY% dengan kelembapan nyata
53    if (isnan(h)) {
54      html.replace("%HUMIDITY%", "--");
55    } else {
56      html.replace("%HUMIDITY%", String(h));
57    }
58
59    // Logika Tombol Dinamis Tunggal
60    int relayState = digitalRead(relayPin);
61    String buttonHtml = "";
62    if (relayState == LOW) {
63      buttonHtml = "<a href='/relay/on'><button class='btn-on'>NYALAKAN (ON)</button></a>";
64    } else {
```

1. Penambahan Sensor Kelembapan (%HUMIDITY%)

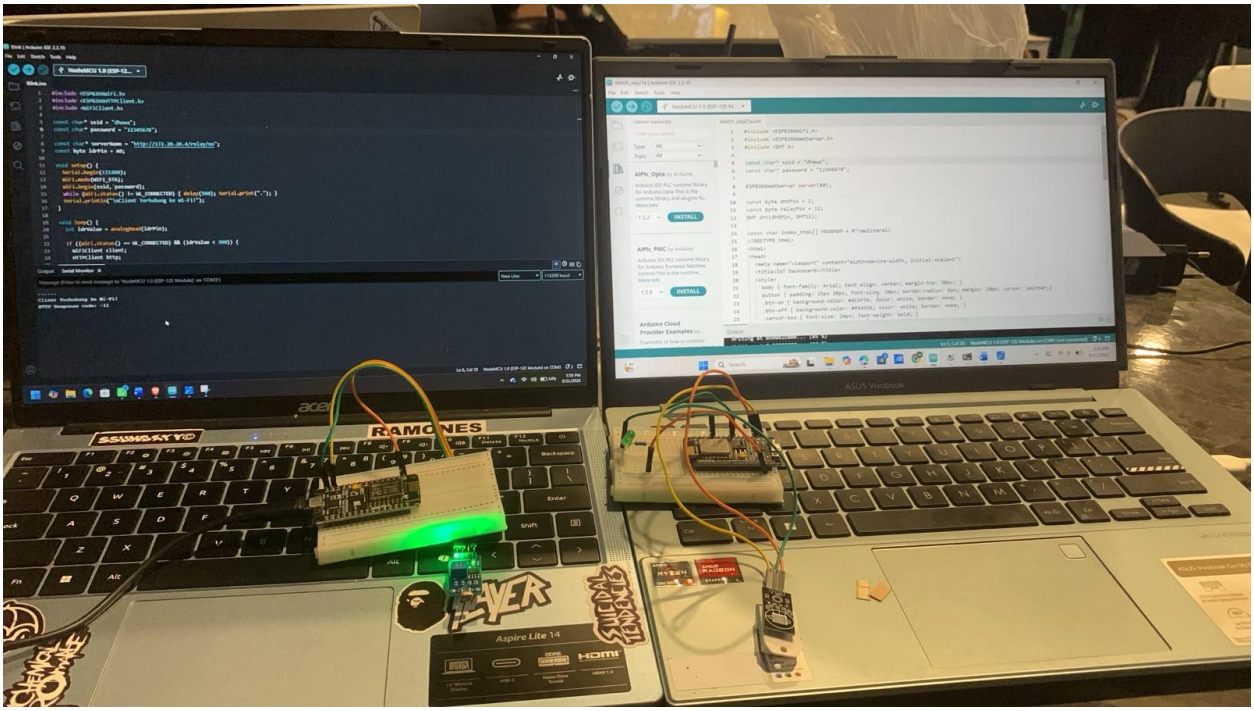
- Konsep: Data kelembapan diambil dari sensor DHT11 menggunakan fungsi bawaan pustaka yaitu dht.readHumidity().
- Implementasi Kode:
    - Pada kerangka HTML (index_html), ditambahkan elemen penampung berupa placeholder %HUMIDITY% di dalam bagian kotak sensor ( ).
    - Di dalam fungsi handleRoot(), nilai pembacaan kelembapan dari sensor disimpan ke dalam variabel float h. Sebelum dikirim ke peramban (*browser*), dilakukan validasi menggunakan fungsi isnan(h) untuk memastikan sensor terbaca dengan benar (tidak mengalami kegagalan hubung).
    - Variabel teks placeholder %HUMIDITY% kemudian diganti menggunakan metode html.replace("%HUMIDITY%", String(h)) agar nilai kelembapan aktual tampil secara dinamis pada halaman web setiap kali halaman dimuat ulang.

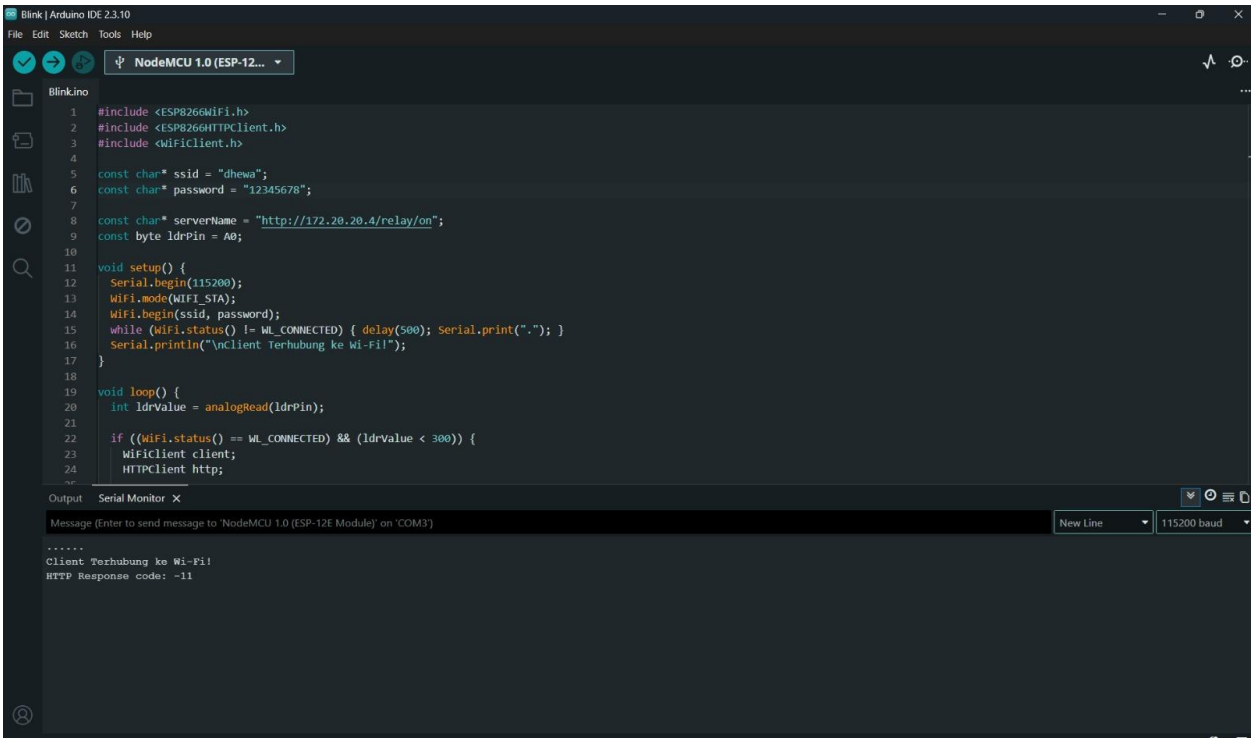
2. Implementasi Tombol Dinamis Tunggal (*Toggle Button*)

- Konsep: Mengubah dua tombol terpisah (tombol ON dan tombol OFF) menjadi hanya satu tombol tunggal yang status tampilannya (warna dan teks) serta fungsi aksinya berubah secara adaptif mengikuti kondisi perangkat keras (status LED/Relay) saat itu.
- Implementasi Kode:
    - Tombol statis pada kerangka HTML dihapus dan diganti dengan placeholder %RELAY_BUTTON%.

- Di dalam fungsi handleRoot(), status fisik pin relay dibaca terlebih dahulu menggunakan fungsi digitalRead(relayPin).
- Melalui blok percabangan if-else:
    - Jika relay berstatus LOW (mati), kode secara dinamis menghasilkan elemen tautan HTML (<a href='/relay/on'>) dengan tombol berwarna hijau bertuliskan "NYALAKAN (ON)".
    - Jika relay berstatus HIGH (nyala), kode secara otomatis mengubah tampilan tautan (<a href='/relay/off'>) dengan tombol berwarna merah bertuliskan "MATIKAN (OFF)".
- Placeholder %RELAY_BUTTON% kemudian diganti dengan string HTML tombol yang telah dikondisikan tersebut.

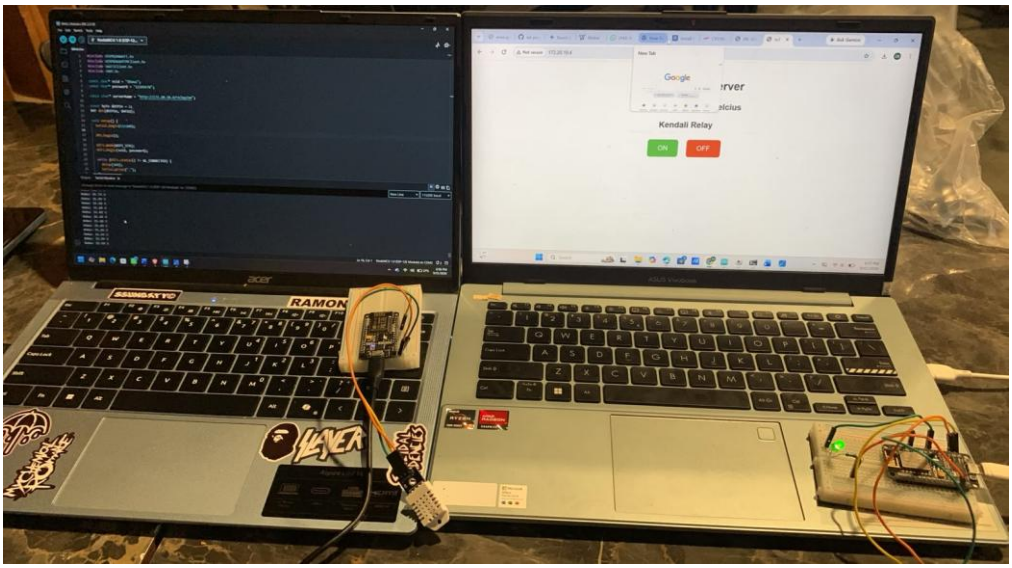
Latihan 3 (Analisis Kode Client): Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).





Hasil Analisis: Pengujian ini dilakukan dengan menjalankan NodeMCU Client dan NodeMCU Server untuk memastikan keduanya dapat berkomunikasi. Setelah itu, NodeMCU Server dimatikan dengan cara mencabut kabel USB atau sumber daya yang terhubung ke Server, sedangkan NodeMCU Client tetap menyala. Selanjutnya Client mencoba mengirim GET Request ke Server yang sudah dalam keadaan offline. Berdasarkan hasil pengujian pada Serial Monitor, Client tetap berhasil terhubung ke Wi-Fi dan menampilkan pesan “Client Terhubung ke Wi-Fi!”, tetapi ketika mencoba berkomunikasi dengan Server, muncul “HTTP Response code: -11”. Hal ini menunjukkan bahwa Client tidak mendapatkan respons HTTP dari Server karena Server dalam keadaan offline.

Latihan 4 (Pengembangan Algoritma): Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.



```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
#include <DHT.h>

const char* ssid = "dhewa";
const char* password = "12345678";

const char* serverName = "http://172.20.10.4/relay/on";

const byte dhtPin = 2;
DHT dht(dhtPin, DHT22);

void setup() {
  Serial.begin(115200);

  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
```

Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
Suhu: 34.70 C
Suhu: 34.80 C
Suhu: 34.80 C
Suhu: 34.90 C
Suhu: 34.90 C
Suhu: 35.00 C
Suhu: 35.00 C
Suhu: 35.00 C
Suhu: 35.10 C
HTTP Response code: 303
Suhu: 35.00 C
Suhu: 35.00 C
```

Bagian yang saya modifikasi



```
#include <DHT.h>

const char* ssid = "dhewa";
const char* password = "12345678";

const char* serverName = "http://172.20.10.4/relay/on";

const byte dhtPin = 2;
DHT dht(dhtPin, DHT22);

void setup() {
  Serial.begin(115200);

  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nClient Terhubung ke Wi-Fi!");
}

void loop() {
  float temperature = dht.readTemperature();

  Serial.print("Suhu: ");
  Serial.print(temperature);
  Serial.println(" C");

  if ((WiFi.status() == WL_CONNECTED) && (temperature > 35)) {
    WiFiClient client;
    HTTPClient http;

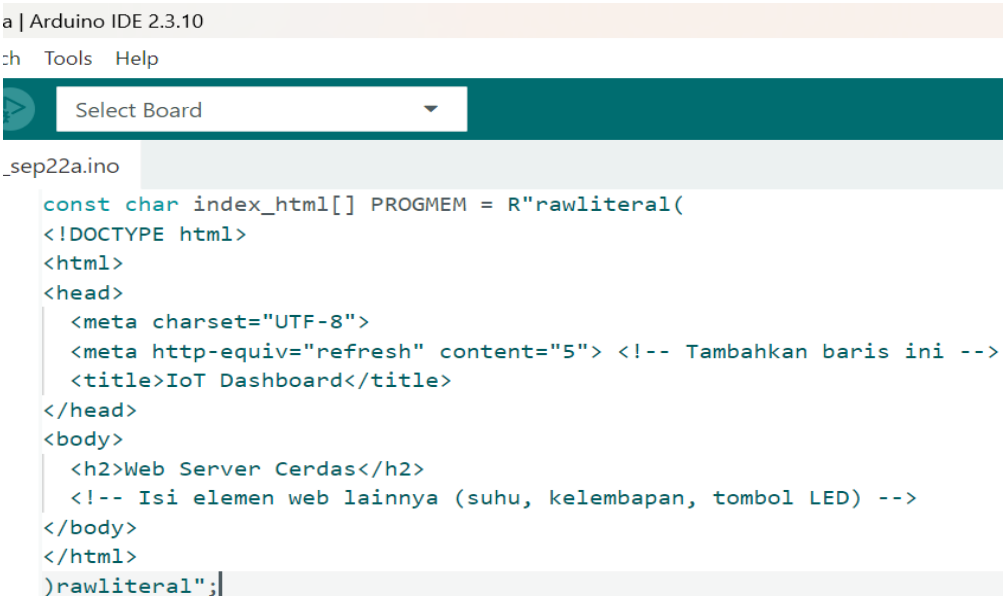
    http.begin(client, serverName);
    int httpResponseCode = http.GET();
```

Modifikasi untuk Latihan 4 dari Praktikum 2 dapat diringkas melalui beberapa poin berikut:

- **Pergantian Sensor:** Sensor LDR digantikan oleh DHT22 khusus untuk mendeteksi suhu.
- **Pustaka & Fungsi:** Menambahkan library DHT.h serta fungsi dht.readTemperature() guna membaca data suhu secara langsung.
- **Aturan Kondisional:** Program menetapkan syarat temperature > 35. *GET Request* hanya dikirim jika suhu di atas 35°C, dan diabaikan jika suhu bernilai 35°C atau lebih rendah.

- **Kesimpulan:** Kendali utama pengiriman perintah ke server sekarang sepenuhnya bergantung pada pembacaan suhu DHT22, menggantikan peran sensor LDR sebelumnya.

3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur



Penambahan kode/memodifikasi

```
<head>
  <meta charset="UTF-8">
  <meta http-equiv="refresh" content="5"> <!-- Tambahkan baris ini -->
  <title>IoT Dashboard</title>
</head>
<body>
  <h2>Web Server Cerdas</h2>
  <!-- Isi elemen web lainnya (suhu, kelembapan, tombol LED) -->
```

Penambahan kode <meta http-equiv="refresh" content="5"> di dalam bagian tag <head> dari variabel HTML berfungsi untuk memerintahkan *browser* klien agar secara otomatis memuat ulang (*reload*) halaman web setiap 5 detik sekali. Dengan menyisipkan instruksi ini, *browser* akan terus mengirimkan permintaan baru secara berkala ke server NodeMCU, sehingga nilai suhu dan kelembapan yang ditampilkan pada antarmuka *dashboard* dapat diperbarui secara otomatis tanpa

pengguna perlu menekan tombol *refresh* atau memuat ulang halaman secara manual.